Lecture Notes: Large Language Models

Course: 2110573 Pattern Recognition

2026-04-09
version 0.1

1 Deep Generative Models and Text Generation

1.1 Distribution Learning Landscape

Generative models fall under the umbrella of distribution learning, which can be broadly classified based on the training strategy:

- **Maximum Likelihood:** Involves explicit likelihood maximization of $P(X)$. It can be further divided into Explicit Density (e.g., Tractable density, Approximate density like Variational Autoencoders) and Implicit Density (e.g., Markov Chain, GSN).
- **Direct Optimization:** Methods like Generative Adversarial Networks (GANs) optimize in ways other than maximizing $P(X)$.

1.2 Text Generation

Unlike other generative tasks (like images where Diffusion models dominate), explicit density methods, specifically **autoregressive prediction**, work exceptionally well for text generation.

- Attempts have been made to use GANs, REINFORCE, Diffusion, and VAEs for text generation, but they are generally not as competitive as simple cross-entropy-based prediction.
- Autoregressive prediction scales very well with large datasets. (Note: recent research is starting to look into using diffusion for text generation).

1.3 Autoregressive Models & LLMs

To generate a sequence of words, the model predicts one word at a time.

- **Training phase:** Training is done by predicting token-by-token to generate a sequence. The previous output is fed back as an input to ensure consistency in generation (the model "knows" what it output before).
- **Teacher Forcing:** During training, the model is fed the ground-truth previous tokens (not its own predictions) to predict the next token.
- The model fundamentally learns the conditional probability: $P(\text{next token} \mid \text{history})$.

2 Pretrained Foundation Models and Emergent Abilities

2.1 Self-Supervised Learning

Foundation models/LLMs are pretrained using self-supervised learning methods such as contrastive learning, next-token prediction, and masked-token prediction. It has been consistently observed that larger models produce better results.

2.2 Emergent Abilities

As language models scale up in size, they demonstrate **emergent abilities**—capabilities that are not present in smaller models but suddenly appear and improve dramatically after a certain size threshold (e.g., modular arithmetic, word unscrambling, Persian QA).

2.3 Why Large Models Matter

In the GPT-3 paper, OpenAI discovered that large models are excellent **few-shot learners** via In-Context Learning.

- **General Model (In-Context Learning):** Instead of traditional fine-tuning (like BERT), large models can perform tasks based purely on the prompt (Task description + Examples → Prompt). This includes Zero-Shot, One-Shot, and Few-Shot settings.
- **Specific Model (Fine-Tuning):** On average, a general model's zero-shot/few-shot accuracy across multiple tasks may not beat a smaller model fine-tuned specifically for a single task (e.g., GPT-3 few-shot is often on par with a fine-tuned BERT-Large).

3 Scaling Laws

The realization that "Scale is all you need" led to efforts to understand the mathematical relationship between data size, compute power, and model performance.

3.1 Kaplan Scaling Law (OpenAI, 2020)

An empirical study treating scaling laws similar to Moore's Law (observed on a log scale).

- Performance depends mostly on scale rather than network hyperparameters like width or depth.
- Data, compute, and network size must be scaled in tandem.
- **Sample Efficiency:** Larger models are more sample-efficient, requiring fewer gradient updates to reach the same performance.
- Models converge slowly; training can be stopped early as late iterations offer diminishing returns.
- Rule of thumb from Kaplan: For a model like GPT-3, you need about 1.7 text tokens per parameter to be efficient.

3.2 Chinchilla Scaling Law (DeepMind, 2022)

DeepMind researchers challenged the Kaplan scaling law, finding that model size and training data should be scaled **equally**.

- The optimal ratio is approximately **20 tokens per parameter**.
- By using a smaller model but training it on significantly more data, DeepMind created *Chinchilla* (70B parameters, 1.4 Trillion tokens), which outperformed the larger *Gopher* (280B parameters, 300 Billion tokens).
- The optimal compute budget allocation can be modeled as:

$$\hat{L}(N, D) \triangleq E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}$$

  Where N is model size, D is data size, E is inherent noise, and A, B, α, β are constants.

4 Alignment and Instruction Tuning

4.1 The Alignment Problem

Self-supervised models are fundamentally "stochastic parrots"—they generate the most statistically probable continuation of a prompt rather than the most helpful one.

- **Alignment Problem:** The challenge of aligning model capabilities with user interests and intents. Supervision is required to fix this.
- **Current Solutions:** Prompt Engineering (manual), Low parameter fine-tuning (e.g., LoRA), and Preference Learning.

4.2 InstructGPT Architecture (OpenAI, 2022)

InstructGPT introduced a multi-step pipeline that forms the basis of modern conversational LLMs like ChatGPT:

1. **Step 0: Pretraining.** Train a base language model (e.g., GPT-3) on vast amounts of text.
2. **Step 1: Supervised Fine-Tuning (SFT).** Also known as Instruction Tuning. Train the model on a curated dataset of questions and high-quality human answers (Instruction Data) using next-word prediction.
3. **Step 2: Reward Model Training.** Train a model to score how good an output is based on human preferences.
4. **Step 3: Reinforcement Learning with Human Feedback (RLHF).** Use the reward model to optimize the SFT model via RL (typically PPO).

4.3 Instruction Data

- **Distillation:** Open-source models often use outputs generated by powerful proprietary models (like ChatGPT) to create their instruction datasets (e.g., Stanford's *Alpaca*).
- **Self-Instruct:** A technique using the LLM itself to generate prompt-response pairs to train more LLMs.
- **Thai Instruction Data:** Examples include OpenThaiGPT and WangchangX (Seed-free synthetic instruct datasets).

5 Preference Learning and RLHF

5.1 Reward Model

We need a way to assign a scalar reward to any input-output pair to represent human judgment.

- **Data Collection:** Annotators write a prompt, the model samples multiple responses, and humans rank them ("chosen" vs "rejected"). Note: LLaMA-3 added an "edited" response where the chosen response is further refined.
- **Contrastive Loss:** The reward model is trained using a contrastive loss function to predict a higher score for the chosen response over the rejected one:

$$\mathcal{L}_R(r_\phi, \mathcal{D}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

- **Heuristics:** For models like DeepSeek R1, math and coding problems act as a natural reward model since the answers can be deterministically verified against a real solution.

5.2 Reinforcement Learning with Human Feedback (RLHF)

The Reward model is used to train the generative policy using Proximal Policy Optimization (PPO).

- **KL Penalty:** A per-token Kullback-Leibler (KL) penalty is added to the objective to ensure the optimized RL model does not deviate too far from the original SFT model.

$$r_t = r_\phi(q, o_{\leq t}) - \beta \log \frac{\pi_\theta(o_t \mid q, o_{<t})}{\pi_{\text{ref}}(o_t \mid q, o_{<t})}$$

- **Why RL?** Supervised learning is inadequate for open-ended creative tasks because you cannot foresee all possible correct answers. RL offers a scalable solution that covers multiple domains and can discover novel, optimal outputs (e.g., AlphaGo's "Move 37" or the "aha moment" in DeepSeek v3).

6 Advanced Preference Learning Techniques

RLHF via PPO is complex and hard to train. Several alternatives have been proposed.

6.1 Direct Preference Optimization (DPO)

DPO skips the reward model training phase entirely and learns directly from the preference data. It is a mathematically derived version of PPO assuming a binary choice in the reward model.

The DPO loss function is given by:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l)\sim\mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right]$$

6.2 Group Relative Policy Optimization (GRPO)

GRPO is an on-policy RL algorithm that calculates the advantage function differently. Standard PPO requires a Value function $V(s)$ to compute the baseline $b(s)$, meaning a separate value model must be trained (which is cumbersome).

GRPO eliminates the need for a separate value network by computing the baseline relative to a group of sampled outputs for the same query. The objective function is:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{q\sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{\text{old}}}(O|q)} \left[\frac{1}{G} \sum_{i=1}^{G} \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left(M_{i,t} \hat{A}_{i,t}, \text{clip}(M_{i,t}, 1-\epsilon, 1+\epsilon) \hat{A}_{i,t} \right) - \beta \mathbb{D}_{KL}[\pi_\theta || \pi_{\text{ref}}] \right\} \right]$$

where $M_{i,t} = \frac{\pi_\theta(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})}$ represents the probability ratio.

The advantage function $\hat{A}_{i,t}$ is standardized across the group of size G:

$$\hat{A}_{i,t} = \frac{r_i - \text{mean}(\{r_1, r_2, \cdots, r_G\})}{\text{std}(\{r_1, r_2, \cdots, r_G\})}$$

6.3 Dr. GRPO / GDPO

GRPO naturally has a bias in its normalization term. By dividing by the length of the response $|o_i|$ and standardizing, it mathematically prefers shorter correct answers while penalizing long incorrect answers less severely. Furthermore, the standard deviation heavily weights questions that are very easy or very hard (where variance is low).

Dr. GRPO fixes this by removing the length penalty ($\frac{1}{|o_i|}$) and the standard deviation scaling from the advantage function:

$$\mathcal{J}_{\text{Dr. GRPO}}(\theta) = \frac{1}{G} \sum_{i=1}^{G} \sum_{t=1}^{|o_i|} \left\{ \min \left(\frac{\pi_\theta(o_{i,t}|\mathbf{q}, \mathbf{o}_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|\mathbf{q}, \mathbf{o}_{i,<t})} \hat{A}_{i,t}, \text{clip}(\dots) \hat{A}_{i,t} \right) \right\}$$

Where the advantage is just the mean-centered reward:

$$\hat{A}_{i,t} = R(\mathbf{q}, \mathbf{o}_i) - \text{mean}(\{R(\mathbf{q}, \mathbf{o}_1), \dots, R(\mathbf{q}, \mathbf{o}_G)\})$$

6.4 On-Policy vs. Off-Policy

- **On-policy:** GRPO and PPO. They learn best from actions based on the current state of the model.
- **Off-policy:** DPO. It relies on fixed preference datasets.

7 RAG and Agentic AI

7.1 Retrieval Augmented Generation (RAG)

Combines a search retriever with an LLM. Given a query, the retriever finds the most relevant documents, and the LLM uses those documents to construct an answer. This drastically reduces hallucinations. (Note: For Thai sparse search, text segmentation is crucial).

7.2 Tools, Skills, and Agents

- **Reasoning vs Non-Reasoning Models:** Models like GPT-o1 and Gemini 2.5 are trained specifically to perform Chain-of-Thought (CoT) planning before answering (reasoning inside '<think>' tags hidden from the user). Using "think step-by-step" prompts on non-reasoning models (GPT-4o) might actually hurt performance.
- **Tool Use:** ChatGPT can output special tokens (e.g., '<search=...>') to pause generation, query an external tool, and inject the result back into the context.
- **Anthropic's MCP (Model Context Protocol):** A standard protocol for LLMs to use outside tools, specifying input/output formats and tool descriptions. Anthropic expanded this into **Skills**, which includes further operational instructions to reduce token overhead.
- **Agentic Workflows:** Single agents suffer from tool overload, context limits, and lack of specialization. *Multi-Agent Systems* resolve this via a Supervisor Agent or Hierarchical Supervisor System that routes queries to specialized sub-agents.

8 Multi-modal LLMs & Compute Efficiency

8.1 Multi-modal LLMs

Modern LLMs handle various modalities (vision, audio, video) by turning them into vectors/tokens and treating them as standard text tokens. Typically trained in 3 steps: 1) Text LLM, 2) Vision/Audio model, 3) Combine. (e.g., Qwen 3.5 natively combines these steps).

8.2 KV Cache and Attention

- In **Causal Attention**, a token only attends to itself and past tokens.
- The Key (K) and Value (V) matrices for previous tokens are cached in memory so they do not need to be recomputed for every new generated token.
- There is no need to store Query (Q) matrices for previous tokens because past tokens no longer need to query the future.

8.3 Prefilling vs Decoding on GPUs

1. **Prefill Phase (Compute Bound):** Processes the initial prompt sequence in parallel across all tokens. Uses high compute core utilization.
2. **Decode Phase (Memory Bound):** Autoregressive token-by-token generation. Relies heavily on memory bandwidth to transfer weights and the KV Cache.

8.4 Cost Reduction Tricks

- **Prompt Caching:** System prompts, RAG documents, and common user prompts can be cached by commercial APIs to drastically lower costs.
- **Cap Thinking Budget:** Limit the maximum reasoning tokens.
- **Model Routing:** Route easy questions to cheaper, smaller models, and hard questions to large reasoning models.
- **Doc2Lora:** Turn static documents directly into model weight updates (LoRA) instead of loading them into the context window.

9 Summary

The lecture provided a comprehensive overview of Large Language Models, synthesizing the core principles required to build and deploy them effectively. The main takeaways include:

- **Scaling Laws:** Understanding how scaling compute, data, and model parameters in tandem reliably improves performance (e.g., Kaplan and Chinchilla laws).
- **ChatGPT Training Pipeline:** The foundational methodology for creating aligned conversational AI:
  - **Pre-training:** Self-supervised learning (next-token prediction) on vast amounts of data to build base capabilities.
  - **Post-training:** Refining the base model for interaction and safety.
  - **Supervised Fine-Tuning (SFT):** Training the model on curated human instructions and demonstrations.
  - **Preference Optimization:** Leveraging alignment techniques like RLHF, DPO, and GRPO/Dr. GRPO to match the model's outputs with human preferences and incentivize advanced capabilities like reasoning.